K L (Deemed to be) University

Department of Computer Science and Engineering

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

KERNEL RESOURCE MONITOR

Linux CPU, RAM, Disk, Process and Load Monitoring

PROJECT DOCUMENTATION

Team	TechNova
M. Subhash	2520030391
D. Namith	2520030249
E. Yeshwanth	2520030539
Academic Year	2026–2027
Course	Operating Systems and Systems Programming

1. Abstract

Kernel Resource Monitor is a lightweight terminal-based monitoring application developed in C for Ubuntu Linux. It reads operating-system resource statistics using the Linux /proc virtual filesystem and the statvfs API. The program calculates sampled CPU utilization, RAM consumption, root filesystem disk usage, process count, and one-minute load average. Readings refresh once per second and warnings appear when CPU, RAM, or disk usage exceeds 80%. A separately labeled demo mode generates simulated readings to illustrate how the dashboard and alerts respond. The program is read-only and runs in user space; it does not modify the Linux kernel.

2. Problem Statement

Linux offers system information through several files and utilities, but checking CPU, memory, storage, and process data separately can be inconvenient for beginners. A small unified tool can present the most important measurements in a clear, continuously refreshed terminal dashboard.

3. Objectives

- Develop a C-based resource monitoring program for Ubuntu.
- Read real system data through /proc and Linux filesystem APIs.
- Show CPU, RAM, disk usage, process count, and load average.
- Update measurements once per second and flag usage above 80%.
- Provide a clearly marked simulated demonstration mode for presentations.

4. Scope

The project monitors the local Linux environment only. It does not monitor other machines, store long-term history, send notifications, or control kernel processes. Total processes means numeric PID directories visible under /proc, not exclusively CPU-running processes.

5. Software and Hardware Requirements

Component	Requirement
Operating system	Ubuntu or compatible Linux distribution
Language	C (C11)
Compiler	GCC
Editor	gedit, VS Code, or any text editor
Interfaces	/proc/stat, /proc/meminfo, /proc/loadavg, /proc, statvfs("/")
Hardware	Any computer or virtual machine capable of running Ubuntu

6. System Architecture

Linux kernel statistics and filesystem counters → C file/API readers → resource calculations → threshold comparison → terminal dashboard → one-second refresh loop.

7. Modules and Implementation

7.1 CPU monitoring

The program reads aggregate CPU counters from /proc/stat twice, one second apart. CPU utilization is computed from the change in total and idle CPU time: CPU% = 100 × (ΔTotal − ΔIdle) / ΔTotal. The first sample initializes the counters.

7.2 Memory monitoring

The program reads MemTotal and MemAvailable from /proc/meminfo. Estimated RAM usage = 100 × (MemTotal − MemAvailable) / MemTotal.

7.3 Disk monitoring

statvfs("/") supplies block counts for the root filesystem. The dashboard computes 100 × (1 − available blocks / total blocks). The value may remain constant across many refreshes.

7.4 Process monitoring

The program scans /proc and counts entries whose directory names contain only digits, representing visible process IDs. The count changes when processes start or exit.

7.5 Load average

The program reads the first value in /proc/loadavg, representing the one-minute load average. This is not a CPU percentage; it summarizes runnable and uninterruptible task demand.

7.6 Alerts

If CPU, RAM, or disk usage exceeds 80%, the program prints a warning for that resource. Otherwise, it prints a normal status message. Process count and load average are displayed but do not trigger alerts in the current implementation.

7.7 Demonstration mode

The --demo argument generates simulated percentages and process counts that change every second. The dashboard explicitly says DEMO (SIMULATED VALUES). This mode illustrates UI behavior and threshold alerts; it is not evidence of actual system measurements.

8. Algorithm

1. Start the application and determine whether --demo was requested.
2. In live mode, initialize the first CPU counter sample.
3. Wait one second.
4. Collect live system statistics, or generate simulated demo statistics.
5. Clear the terminal and print labeled measurements.
6. Compare CPU, RAM, and disk readings with the 80% threshold.

7. Print warnings or a normal status message.

8. Repeat until the user presses Ctrl+C.

9. Compilation and Execution

Save the source as KernelMonitor.c in the project folder. Open Terminal in that folder and run:

gcc -Wall -Wextra -std=c11 KernelMonitor.c -o KernelMonitor

./KernelMonitor

./KernelMonitor --demo

The first command compiles the program. The second starts real Ubuntu monitoring. The third starts simulated demonstration mode. Press Ctrl+C to stop either mode.

10. Sample Output

========== KERNEL RESOURCE MONITOR ==========
MODE: LIVE (REAL UBUNTU DATA)
Refresh: 1 second | Update: 22
CPU usage : 0.00%
RAM usage : 7.31%
Disk usage : 5.33%
Total processes : 35
1-minute load : 0.00
[NORMAL] No threshold exceeded
Press Ctrl+C to stop.

The sample above was provided from the project run. Values depend on the system and may not change on every refresh. A 0.00 load reading can represent a very low value rounded to two decimal places.

11. Testing and Validation

Test	Expected behavior
Compilation	GCC builds the source without errors
Live mode	Shows actual Ubuntu readings with LIVE label
Refresh	Update counter increases approximately once per second
Process test	Starting a temporary process can change the count
Demo mode	SIMULATED label appears and readings change
Threshold alert	Demo readings above 80% display warnings
Exit	Ctrl+C terminates monitoring

12. Results and Discussion

The program consolidates five system measurements in one terminal view and performs periodic threshold checks. CPU and RAM may fluctuate; disk utilization and process counts can remain stable. Load average changes gradually because it is a smoothed system metric. Demo mode is useful for demonstrating alerts but must not be presented as live evidence.

13. Limitations

- Works only on Linux systems exposing the required interfaces.
- No persistent logging, graphs, remote monitoring, or notifications.
- A fixed 80% alert threshold is compiled into the source.
- Process count is a snapshot and does not indicate how many processes are actively on CPU.
- Demo values are synthetic and should not be used to evaluate real system health.

14. Future Enhancements

- Allow user-configurable thresholds.
- Save measurements to CSV and plot historical trends.
- Add desktop notifications for sustained high usage.
- Display per-process CPU and RAM utilization.
- Add monitoring of multiple disks or network statistics.

15. Conclusion

Kernel Resource Monitor demonstrates practical Linux systems programming in C by reading kernel-provided statistics, processing resource measurements, and presenting the results in a live command-line interface. The project supports introductory learning about CPU scheduling statistics, memory availability, process management, and filesystem monitoring.

16. References

- Linux proc(5), proc_stat(5), proc_meminfo(5), proc_loadavg(5) manual pages.
- Linux statvfs(3) and readdir(3) manual pages.
- KernelMonitor.c source file supplied with the TechNova project.

Appendix A — Source Code

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <ctype.h>
#include <unistd.h>
#include <time.h>
#include <sys/statvfs.h>
```

```
double cpu_usage(void) {
    static unsigned long long prev_total=0, prev_idle=0;
    FILE *f=fopen("/proc/stat","r");
    unsigned long long u,n,s,i,w,irq,soft,steal;
    if (!f) return -1;
    int ok=fscanf(f,"cpu %llu %llu %llu %llu %llu %llu %llu %llu",
        &u,&n,&s,&i,&w,&irq,&soft,&steal);
    fclose(f);
    if(ok!=8) return -1;
    unsigned long long idle=i+w, total=u+n+s+i+w+irq+soft+steal;
    if(!prev_total) {prev_total=total;prev_idle=idle;return -1;}
    unsigned long long dt=total-prev_total,di=idle-prev_idle;
    prev_total=total;prev_idle=idle;
    return dt ? 100.0*(dt-di)/dt : 0;
}
double ram_usage(void) {
    FILE *f=fopen("/proc/meminfo","r");
    if(!f) return -1;
    char line[256]; unsigned long total=0,avail=0;
    while(fgets(line,sizeof line,f)) {
        if(strncmp(line,"MemTotal:",9)==0) sscanf(line,"MemTotal: %lu",&total);
        if(strncmp(line,"MemAvailable:",13)==0) sscanf(line,"MemAvailable: %lu",&avail);
    }
    fclose(f);
    return total && avail<=total ? 100.0*(total-avail)/total : -1;
}
double disk_usage(void) {
    struct statvfs d;
    if(statvfs("/",&d)!=0 || !d.f_blocks) return -1;
    return 100.0*(1.0-(double)d.f_bavail/d.f_blocks);
}
int process_count(void) {
    DIR *d=opendir("/proc"); if(!d) return -1;
    struct dirent *e; int count=0;
    while((e=readdir(d))) {
        const char *p=e->d_name;
        if(!*p) continue;
        for(;*p && isdigit((unsigned char)*p);p++);
        if(!*p) count++;
    }
    closedir(d); return count;
}
double load_average(void) {
    FILE *f=fopen("/proc/loadavg","r"); double x;
    if(!f) return -1;
    int ok=fscanf(f,"%lf",&x); fclose(f);
    return ok==1 ? x : -1;
}
double demo_percent(int low,int high) {
    return low+(rand()%((high-low)*100+1))/100.0;
}
void print_metric(const char *label,double value,const char *suffix) {
    if(value<0) printf("%-18s: Unavailable\n",label);
    else printf("%-18s: %.2f%s\n",label,value,suffix);
}
int main(int argc,char **argv) {
    int demo=argc==2 && strcmp(argv[1],"--demo")==0;
    if(argc>2 || (argc==2 && !demo)) {
        fprintf(stderr,"Usage: %s [--demo]\n",argv[0]); return 1;
    }
    srand((unsigned)time(NULL));
```

```
    if(!demo) cpu_usage();
    unsigned long tick=0;
    for(;;) {
        sleep(1);
        double cpu=demo?demo_percent(10,95):cpu_usage();
        double ram=demo?demo_percent(30,92):ram_usage();
        double disk=demo?demo_percent(20,90):disk_usage();
        int processes=demo?30+rand()%41:process_count();
        double load=demo?demo_percent(0,4):load_average();
        printf("\033[H\033[J");
        printf("=========== KERNEL RESOURCE MONITOR ===========\n");
        printf("MODE: %s\n",demo?"DEMO (SIMULATED VALUES)":"LIVE (REAL UBUNTU DATA)");
        printf("Refresh: 1 second | Update: %lu\n",++tick);
        printf("-----------------------------------------------\n");
        print_metric("CPU usage",cpu,"%");
        print_metric("RAM usage",ram,"%");
        print_metric("Disk usage",disk,"%");
        if(processes>=0) printf("%-18s: %d\n","Total processes",processes);
        else printf("Total processes   : Unavailable\n");
        print_metric("1-minute load",load,"");
        printf("-----------------------------------------------\n");
        int warn=0;
        if(cpu>80) {puts("[WARNING] High CPU usage");warn=1;}
        if(ram>80) {puts("[WARNING] High RAM usage");warn=1;}
        if(disk>80) {puts("[WARNING] High disk usage");warn=1;}
        if(!warn) puts("[NORMAL] No threshold exceeded");
        puts("Press Ctrl+C to stop.");
        fflush(stdout);
    }
}
```